

16. Glossario

16. Glossario

Questo glossario raccoglie in un unico posto tutti i termini incontrati nel corso, dai concetti base di informatica fino alla sicurezza e agli ambienti di produzione. Le voci sono in **ordine alfabetico rigoroso** (gli acronimi come CI, CPU o KPI sono ordinati per le lettere dell'acronimo stesso, non per il nome per esteso), così puoi usarlo come un vero dizionario: se leggi un termine che non ricordi mentre studi una sezione, torna qui, cercalo e poi eventualmente segui il rimando “→ approfondito nella sezione X” per tornare al capitolo che lo spiega con esempi e contesto completo.

Non serve leggerlo tutto d'un fiato: è pensato per essere consultato, non studiato in ordine. Buona consultazione.



Mappa dei concetti: come si collegano

Questo glossario ha due viste, da usare per scopi diversi: la mappa qui sotto ti mostra **come i termini si concatenano tra loro**, raggruppati per famiglia tematica, mentre l'elenco alfabetico che segue resta il dizionario da consultare voce per voce quando ti serve una definizione precisa. Le definizioni non si ripetono qui: sotto trovi solo i **collegamenti**.

Le basi tecniche

Tutto parte dal **Computer**: un insieme di componenti hardware in cui la **CPU** esegue le istruzioni usando la **RAM** come memoria di lavoro temporanea e il **Disco** come memoria permanente, organizzata in cartelle e file grazie al **File System**. Il **Sistema Operativo** fa da intermediario tra questo hardware e i programmi, e ogni programma in esecuzione diventa un **Processo**, a sua volta eventualmente suddiviso in più **Thread** che lavorano in parallelo. Su questa base tecnica gira tutto il resto: dal browser di un cliente di ShopFacile al server che elabora i suoi ordini.

Come le applicazioni comunicano

Il modello **Client-Server** descrive chi chiede (client) e chi risponde (server); per farlo, i due capi si scambiano dati attraverso una **Rete**, seguendo le regole del **TCP/IP** e, nello specifico del web, del protocollo **HTTP** (o della sua versione cifrata **HTTPS**). Prima ancora di parlarsi, il client deve sapere *dove* si trova il server: è compito del **DNS** tradurre un nome leggibile in un indirizzo. Una volta stabilita la comunicazione, i programmi si scambiano informazioni tramite una **API**, spesso progettata secondo lo stile **REST** e con i dati serializzati in **JSON** (o, più raramente oggi, **XML**).

Come si struttura un'applicazione

Un'applicazione divide le proprie responsabilità tra **Frontend** (quello che l'utente vede) e **Backend** (la logica e i dati che l'utente non vede); quest'ultimo può essere organizzato come un unico **Monolite** oppure spezzato in **Microservizi** indipendenti che comunicano tra loro, a volte tramite una **Message Queue** invece che con chiamate dirette. Un'alternativa a gestire server propri è il modello **Serverless**, dove il codice viene eseguito solo su richiesta.

Dove vivono i dati

I dati di ShopFacile si salvano in un **Database relazionale**, interrogabile con **SQL**, oppure in un **Database NoSQL** quando la struttura dei dati è più flessibile o i volumi molto grandi: la scelta tra i due dipende

dalla forma dei dati (es. il catalogo prodotti rispetto ai log delle sessioni carrello), ma anche dal **CAP Theorem**, che spiega quale compromesso tra coerenza e disponibilità ciascun database privilegia quando la rete ha un problema. Quando un database da solo non basta più, entrano in gioco due strategie complementari: la **Replication** (più copie per scalare le letture) e lo **Sharding** (dati divisi tra più macchine per scalare volumi e scritture). Per evitare di interrogare il database ad ogni richiesta, si usa spesso una **Cache-Aside (lazy loading)**, con un **TTL** che ne scandisce la scadenza.

Come si scrive e si versiona il codice

Tutto il codice vive in un **Repository**, la cui storia è fatta di **Commit** successivi, ciascuno registrato su un **Branch** dedicato per non toccare il codice principale durante lo sviluppo. Quando una modifica è pronta, si apre una **Pull Request**: qui il team fa **Code Review** prima di eseguire il **Merge** che la unisce definitivamente. Un punto del repository può poi essere marcato con un **Tag**, tipicamente per identificare una **Release**, seguendo una convenzione di **Versioning** come il **Semantic Versioning**. Il modo in cui i branch vengono organizzati nel tempo segue un modello come il **Git Flow** oppure, all'opposto, il **Trunk Based Development**; un problema scoperto lungo il percorso viene tracciato come **Issue**.

Come nasce e cresce un prodotto

Tutto comincia dai **Requisiti (funzionali e non funzionali)** che descrivono cosa deve fare il software: da lì nasce una **User Story** che lo racconta dal punto di vista dell'utente, oppure, se il lavoro è troppo grande per uno sprint, un **Epic** che raggruppa più User Story. Ogni User Story diventa una **Feature** una volta realizzata, oppure genera un **Bug** se qualcosa non funziona come previsto. Tutto questo percorso, dalla richiesta iniziale alla manutenzione, è descritto dal **Ciclo di vita del software**.

Come si organizza il lavoro

Il **Manifesto Agile** e il **Mindset Agile** superano il **Waterfall** tradizionale con due framework operativi concreti. Nello Scrum, il **Product Owner** cura il **Product Backlog**, da cui il team seleziona in **Sprint Planning** le voci dello **Sprint Backlog** per ogni **Sprint**, sincronizzandosi nel **Daily Scrum** e chiudendo con la **Sprint Review** e la **Sprint Retrospective**; il risultato è un **Increment** che rispetta la **Definition of Done (DoD)** (ed è entrato in sprint solo se soddisfaceva la **Definition of Ready (DoR)**), mentre lo **Scrum Master** facilita tutto questo. Nel Kanban lo stesso lavoro scorre su una **Board Kanban** con il

WIP (Work In Progress) limitato per colonna, misurato in **Lead Time** e **Cycle Time** (lo **Scrumban** mescola i due mondi); gli **Story Point** e la **Velocity** quantificano la capacità del team di Luca, mentre **KPI**, **Milestone**, **RACI**, **RAID Log**, il **Rischio** e gli **Stakeholder**, insieme a **Burndown Chart** e **Burnup Chart**, tengono sotto controllo l'andamento del progetto nel suo complesso.

Come si governa un progetto (il vocabolario PMP)

Il **PMI (Project Management Institute)** raccoglie nel **PMBOK Guide** il vocabolario tradizionale del Project Management, alla base della certificazione **PMP (Project Management Professional)**: tutto parte dal **Project Charter**, il documento con cui lo **Sponsor** autorizza formalmente il progetto, da cui si scompone lo **Scope (ambito)** in una **WBS (Work Breakdown Structure)** per capire tutto il lavoro da fare. Da qui si costruisce la schedulazione, individuando il **Critical Path (percorso critico)** e rappresentandolo in un **Diagramma di Gantt**, mentre l'avanzamento viene controllato con l'**EVM (Earned Value Management)** attraverso gli indici **SPI (Schedule Performance Index)** e **CPI (Cost Performance Index)**, sempre nel rispetto del **Triplo vincolo (triple constraint)** tra tempo, costo e ambito. Ogni variazione passa da una **Change Request (richiesta di modifica)** formale, proprio per evitare lo **Scope Creep**; a fine progetto, le **Lessons Learned** chiudono il cerchio, raccogliendo cosa ha funzionato e cosa no. Questo vocabolario più formale — pensato originariamente per progetti pianificati per fasi — si sovrappone e convive con quello Agile visto sopra (**Product Backlog**, **Sprint Retrospective**, **Velocity**): nella pratica quotidiana di molti team, incluso quello di ShopFacile, i due mondi si combinano in un **Approccio ibrido (hybrid)**, usando gli strumenti PMP per la visione d'insieme e di lungo periodo e Scrum/Kanban per il lavoro operativo di ogni sprint.

Come il codice arriva agli utenti

Quando una Pull Request supera la Code Review ed entra nel branch principale, un **Trigger** avvia la **Pipeline**: build, test e poi il **Quality Gate**, il controllo automatico che blocca l'avanzamento se la qualità non basta. Se tutto va bene, la pipeline produce un **Artifact (build)** che viene promosso dall'**Ambiente di Sviluppo** all'**Ambiente di Test/QA**, poi allo **Staging** e infine alla **Produzione** tramite il **Deploy/Rilascio**; le pratiche di **CI (Continuous Integration)** e **Continuous Delivery/Continuous Deployment** sono ciò che rende questo percorso frequente e automatico invece che manuale e raro. Se qualcosa va storto dopo il rilascio, il **Rollback** riporta rapidamente alla versione precedente.

Dove gira il software

Un Artifact, per essere eseguito ovunque allo stesso modo, viene spesso pacchettizzato in un **Container** tramite **Docker**, e quando i container sono troppi da gestire a mano entra in gioco **Kubernetes**, che li orchestra automaticamente. Un'alternativa più tradizionale è la **Virtual Machine (VM)**, che simula un intero computer; la capacità di questi ambienti di gestire più carico segue i due modi descritti da **Scalabilità verticale e orizzontale**. Tutto questo gira su infrastrutture cloud offerte secondo tre livelli di responsabilità crescente per il fornitore — **IaaS**, **PaaS** e **SaaS** — quasi sempre pagate a consumo con il modello **Pay-as-you-go**.

Come si tiene tutto in piedi e sicuro

Una volta in produzione, il **Monitoring** osserva lo stato del sistema e il **Logging** ne registra gli eventi; l'**Observability** va oltre entrambi, aiutando a capire *perché* qualcosa non funziona, non solo *cosa*. Sul fronte sicurezza, l'**Autenticazione** verifica chi sei e l'**Autorizzazione** stabilisce cosa puoi fare, spesso rafforzate da **MFA (autenticazione a più fattori)**; le vulnerabilità più comuni sono catalogate da **OWASP**, mentre il **GDPR** impone regole precise sul trattamento dei dati personali degli utenti di ShopFacile. Il **DevSecOps** integra questi controlli di sicurezza direttamente nella pipeline, invece di lasciarli come verifica finale isolata.

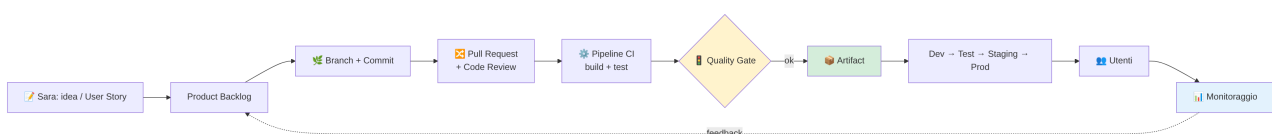
La cultura che tiene insieme tutto

Il **DevOps** è la cornice culturale che tiene insieme tutti i fili visti sopra: il **CALMS** ne riassume i cinque pilastri, mentre la **Cultura DevOps** descrive concretamente cosa cambia nel modo di lavorare quotidiano di un team come quello di ShopFacile, fino a pratiche come l'**IaC (Infrastructure as Code)** che trattano l'infrastruttura come se fosse codice, versionabile e revisionabile come qualsiasi Pull Request.

Come l'AI entra nel lavoro del team

Tutto parte dai dati: attraverso il **Training (addestramento)**, un algoritmo di **Machine Learning** (o, più nello specifico, di **Deep Learning**) impara pattern da grandi quantità di esempi e produce un **Modello (AI)**, che poi viene interrogato in fase di **Inferenza** per generare risposte su casi nuovi. Gli **LLM (Large Language Model)** sono uno dei risultati più visibili di questo processo, e alimentano la categoria dell'**AI generativa**: strumenti concreti come **Copilot (GitHub Copilot)** nascono proprio da qui, trasformando un **Prompt** scritto dallo sviluppatore in suggerimenti di codice. Questa tecnologia ha però dei limiti che il team di ShopFacile deve conoscere prima di fidarsene ciecamente: l'**Allucinazione (AI)**, il **Bias** ereditato dai dati di training, e l'**Overfitting**, quando un modello impara troppo bene gli esempi visti e generalizza male su quelli nuovi. Proprio come il **DevOps** ha portato automazione e collaborazione al rilascio del software, l'**MLOps** ne applica gli stessi principi al ciclo di vita dei modelli AI, mentre l'**AIops** usa queste stesse tecniche per potenziare il monitoraggio dei sistemi in produzione. Un **LLM (Large Language Model)** da solo, però, non conosce i documenti interni di un'azienda come ShopFacile né può agire sui suoi sistemi: il **RAG (Retrieval-Augmented Generation)** risolve il primo limite recuperando al momento della domanda i pezzi (**Chunk**) di documentazione più pertinenti da un **Database vettoriale**, individuati tramite **Ricerca semantica** sugli **Embedding**, invece di dover riaddestrare il modello con il **Fine-tuning**; il **MCP (Model Context Protocol)**, con i suoi **MCP server**, risolve il secondo, dando allo stesso modello un modo standard per collegarsi a strumenti e sistemi reali. In sintesi: il primo dà al modello la conoscenza, il secondo gli dà la capacità di agire.

Il percorso end-to-end in un unico schema



ACID

L'insieme di quattro proprietà che una transazione di database deve garantire: Atomicity (tutto o nulla), Consistency (il database resta in uno stato valido), Isolation (le transazioni concorrenti non si vedono a metà strada) e Durability (una modifica confermata resta anche dopo un crash). → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **MVCC** (la tecnica con cui i database moderni implementano l'isolamento senza bloccare tutto) e **Livello di isolamento** (che stabilisce quanto rigorosamente viene garantita l'isolation).

Active Directory (AD)

Il servizio più diffuso nelle aziende enterprise per gestire in un unico posto l'elenco di utenti, gruppi, computer e politiche di sicurezza, invece di farli gestire a ogni applicazione per conto proprio; parla tipicamente il protocollo LDAP per farsi interrogare dalle altre applicazioni. → approfondito nella sezione 13 (Sicurezza). Si collega a: **LDAP** (il protocollo con cui viene interrogato) e **SSO (Single Sign-On)** (il meccanismo che tipicamente si costruisce sopra di esso).

ADR (Architecture Decision Record)

Un documento breve, uno per ogni decisione tecnica rilevante, scritto nel momento in cui la decisione viene presa (non ricostruito a posteriori) e versionato nel repository accanto al codice: serve a non perdere il **perché** di una scelta, non solo il **cosa**. Un ADR non si cancella né si riscrive: se una decisione viene ripensata, si scrive un nuovo ADR che dichiara di superare il precedente. → approfondito nella sezione 3 (Come nasce un software).

AI (Intelligenza Artificiale)

Sigla di Artificial Intelligence: la capacità di un sistema informatico di svolgere compiti che normalmente richiederebbero l'intelligenza umana, come riconoscere immagini, capire il linguaggio o generare testo. → approfondito nella sezione 15 (Intelligenza artificiale).

AI generativa

Un tipo di intelligenza artificiale capace di creare contenuti nuovi (testo, immagini, codice) invece di limitarsi a classificare o prevedere dati esistenti; è la categoria a cui appartengono strumenti come ChatGPT o GitHub Copilot. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **LLM (Large Language Model)** (la tecnologia che più spesso la rende possibile) e **Copilot (GitHub Copilot)** (uno strumento concreto che la usa).

AIOps

L'uso di tecniche di intelligenza artificiale e machine learning per automatizzare il monitoraggio, l'analisi dei log e la diagnosi dei problemi in un sistema IT complesso, individuando anomalie più rapidamente di un controllo puramente manuale. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Monitoring** (che potenzia con tecniche di AI) e **MLOps** (di cui condivide la logica di automazione, applicata qui alle operations invece che al ciclo di vita del modello).

Allucinazione (AI)

Una risposta di un modello di intelligenza artificiale che sembra sicura e ben scritta ma è in realtà falsa o inventata, ad esempio citare una funzione di libreria che non esiste. È uno dei motivi per cui l'output di uno strumento AI va sempre verificato da una persona, non accettato automaticamente. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Modello (AI)** (che la produce) e **Training (addestramento)** (i cui limiti e le cui lacune sono spesso all'origine del problema).

Ambiente di Sviluppo

L'ambiente (development environment) in cui gli sviluppatori scrivono e provano il codice per la prima volta, di solito sul proprio computer o in uno spazio dedicato non visibile agli utenti finali. È il primo “gradino” prima che il codice arrivi al Test/QA, allo Staging e infine alla Produzione. → approfondito nella sezione 14 (Ambienti di sviluppo).

Ambiente di Test/QA

Un ambiente separato da quello di sviluppo, usato per verificare (Quality Assurance) che una funzionalità funzioni correttamente prima di mostrarla a chiunque altro. Qui si eseguono i test manuali e automatici in condizioni più simili a quelle reali. → approfondito nella sezione 14 (Ambienti di sviluppo). Si collega a: **Ambiente di Sviluppo** (che lo precede) e **Staging** (che ne consegue, prima della Produzione).

API

Sigla di Application Programming Interface: è un insieme di regole che permette a due programmi di “parlarsi” e scambiarsi informazioni, senza che uno debba conoscere i dettagli interni dell'altro. Pensala come un menu di un ristorante: tu scegli una voce (una richiesta) e la cucina (il sistema) ti restituisce il piatto (la risposta). → approfondito nella sezione 2 (Fondamenti di informatica).

Approccio ibrido (hybrid)

Un modo di gestire un progetto che unisce elementi del Project Management tradizionale (quello descritto da PMP e PMBOK Guide) con le pratiche Agile viste nelle sezioni precedenti, usando lo strumento più adatto a ogni situazione invece di seguire un solo metodo in modo rigido. → approfondito nella sezione 8 (Project Management). Si collega a: **PMBOK Guide** (il vocabolario tradizionale) e **Mindset Agile** (il vocabolario iterativo con cui qui convive).

Artifact (build)

Il “prodotto finito” generato da una pipeline dopo aver compilato e testato il codice: ad esempio un file eseguibile, un pacchetto installabile o un'immagine Docker pronta per essere distribuita. È diverso da un servizio come GitHub Packages, che conserva pacchetti e librerie riutilizzabili invece del singolo prodotto di una pipeline. → approfondito nella sezione 10 (CI/CD). Si collega a: **Pipeline** (che lo produce) e **Deploy/Rilascio** (che lo installa, identico, in ogni ambiente).

Autenticazione

Il processo con cui un sistema verifica che tu sia davvero chi dici di essere, tipicamente chiedendo username e password (authentication). È il primo passo della sicurezza: prima si verifica l'identità, poi si decide cosa quella identità può fare. → approfondito nella sezione 13 (Sicurezza).

Autorizzazione

Il processo che stabilisce, dopo che sei stato riconosciuto (autenticazione), a quali risorse puoi accedere e quali azioni puoi compiere (authorization). Esempio: essere autenticati come dipendenti non significa automaticamente poter modificare i dati contabili dell'azienda. → approfondito nella sezione 13 (Sicurezza).

Backend

La parte “dietro le quinte” di un'applicazione: server, database e logica di business che l'utente non vede direttamente, ma che elabora le richieste e restituisce i risultati al Frontend. → approfondito nella sezione 11 (Architetture software).

Batch (elaborazione batch) / Farm batch

L'elaborazione di grandi volumi di dati “a lotti”, tutti insieme e secondo una schedulazione (a un orario fisso o dopo un evento), invece che uno alla volta in risposta a una richiesta dell'utente: tipica del lavoro notturno di un'assicurazione (calcolo premi, produzione documenti, invii a enti esterni). La farm batch è l'insieme delle macchine dedicate a farlo girare, spesso separate dai server che servono il sito o l'app. → approfondito nella sezione 9 (DevOps). Si collega a: **ITSM** (di cui condivide l'attenzione operativa) e **Disaster Recovery** (rilevante se la farm batch è ferma per un guasto).

BIA (Business Impact Analysis)

L'analisi che stima, processo per processo, quanto costa un'ora di fermo in senso di business (non tecnico): quanti ordini non confermati, quanti clienti persi, quale obbligo normativo non rispettato. È il numero che determina quale RTO e quale RPO ha senso concordare per ciascun sistema: una decisione di business, non una scelta tecnica lasciata al team IT. → approfondito nella sezione 13 (Sicurezza). Si collega a: **Business Continuity** (il piano che la BIA aiuta a costruire) e **Disaster Recovery** (i cui RTO/RPO la BIA determina).

Bias

Una distorsione sistematica nelle risposte di un modello di intelligenza artificiale, che nasce quasi sempre dai dati con cui è stato addestrato (ad esempio se quei dati rappresentano in modo sbilanciato certi gruppi o scenari) e che il team deve conoscere per usare questi strumenti in modo responsabile. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (la fase in cui più spesso si origina) e **Modello (AI)** (che lo eredita e lo riproduce nelle risposte).

Board Kanban

Una lavagna (fisica o digitale) divisa in colonne — tipicamente “Da fare”, “In corso”, “Fatto” — che rende visibile lo stato di avanzamento di ogni attività del team con dei cartellini (le card). → approfondito nella sezione 7 (Kanban).

Branch

Una “linea di sviluppo” separata all'interno di un repository Git, che permette di lavorare su una modifica senza toccare il codice principale finché non è pronta. → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Repository** (che lo contiene) e **Pull Request** (che ne propone l'unione al codice principale).

Branching

La pratica di creare e gestire più Branch per isolare il lavoro su funzionalità diverse, permettendo a più persone di lavorare in parallelo senza intralciarsi. → approfondito nella sezione 3 (Come nasce un software).

Bug

Un errore nel software che produce un comportamento diverso da quello previsto: dal termine inglese per “insetto”, nato da un aneddoto storico legato a un vero insetto trovato in un computer degli anni '40. → approfondito nella sezione 3 (Come nasce un software).

Builder image

La prima fase di un multi-stage build: un'immagine Docker “temporanea” che contiene tutti gli strumenti necessari a compilare l'applicazione (compilatore, dipendenze di sviluppo), ma che non finisce mai in produzione — solo il risultato della compilazione viene copiato nell'immagine finale, più piccola e senza gli strumenti di build. → approfondito nella sezione 10 (CI/CD). Si collega a: **Multi-stage build** (la tecnica che la usa) e **Artifact (build)** (il prodotto finale, più leggero, che ne deriva).

Burndown Chart

Un grafico che mostra quanto lavoro **resta** da fare in uno Sprint (o in un progetto) man mano che passano i giorni: la linea idealmente “scende” verso lo zero. → approfondito nella sezione 8 (Project Management).

Burnup Chart

Un grafico simile al Burndown Chart, ma che mostra il lavoro **già completato** che “sale” verso il totale previsto, rendendo più facile notare se l'ambito del progetto (lo scope) cresce nel tempo. → approfondito nella sezione 8 (Project Management).

Business Continuity (BC)

La disciplina che risponde alla domanda “come continua a funzionare l'azienda mentre i sistemi non ci sono?”, distinta dal Disaster Recovery che risponde invece a “come rimetto in piedi i sistemi?”. Include procedure alternative per le persone (cosa fa il call center quando il sistema è giù), comunicazione di crisi e test periodici del piano. → approfondito nella sezione 13 (Sicurezza). Si collega a: **Disaster Recovery** (il piano tecnico complementare) e **BIA (Business Impact Analysis)** (l'analisi che ne guida le priorità).

Cache-Aside (lazy loading)

La strategia di caching più diffusa: l'applicazione controlla prima la cache; se il dato c'è (cache hit) lo restituisce subito senza toccare il database, se non c'è (cache miss) lo recupera dal database, lo salva in cache per la prossima volta, e poi risponde. È semplice e “fallisce bene”: se la cache non è disponibile, il sistema continua a funzionare appoggiandosi al database, solo più lentamente. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **TTL (Time To Live)** (una delle tecniche più comuni per invalidare i dati salvati con questa strategia).

Cactus (modello di branching)

Un modello di branching Git, noto anche come OneFlow, che si colloca tra Git Flow e Trunk Based Development: un solo tronco principale a lungo termine, branch di feature brevi, e branch di release solo quando serve davvero gestire più versioni in produzione contemporaneamente. → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Git Flow** (il modello più strutturato da cui si differenzia) e **Trunk Based Development** (il modello più semplice all'altro estremo).

CAP Theorem

Un teorema che afferma che, in un sistema distribuito, si possono garantire solo due delle tre proprietà Consistency (tutti i nodi vedono gli stessi dati), Availability (il sistema resta sempre disponibile) e Partition Tolerance (il sistema funziona anche con la rete interrotta tra i nodi): dato che la partition tolerance non è quasi mai opzionale in un sistema reale, la scelta pratica è tra Consistency e Availability. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Database NoSQL** (i cui diversi prodotti fanno scelte diverse su questo teorema) e **Sharding** (un'altra strategia tipica dei sistemi distribuiti a cui il teorema si applica).

CALMS

Un acronimo (Culture, Automation, Lean, Measurement, Sharing) che riassume i cinque pilastri della cultura DevOps: cultura collaborativa, automazione, approccio lean, misurazione dei risultati e condivisione della conoscenza. → approfondito nella sezione 9 (DevOps).

Change Request (richiesta di modifica)

Una proposta formale di modificare qualcosa già deciso nel progetto — ambito, tempi, costi o requisiti — che va valutata e approvata prima di essere applicata, invece di lasciare che il progetto cambi rotta senza controllo. → approfondito nella sezione 8 (Project Management). Si collega a: **Scope Creep** (il rischio che corre un progetto se le richieste di modifica non passano da qui).

Chiave esterna (foreign key)

Una colonna (o gruppo di colonne) di una tabella che contiene lo stesso valore della chiave primaria di un'altra tabella, creando così una relazione tra le due; il database la fa rispettare attivamente, rifiutando di creare righe che puntano a un valore inesistente (integrità referenziale). → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Chiave primaria** (a cui punta) e **JOIN** (l'operazione SQL che la usa per unire le due tabelle).

Chiave primaria (primary key)

Una colonna (o un piccolo gruppo di colonne) che identifica in modo univoco e stabile ogni riga di una tabella, e che il database si impegna a non far duplicare mai: deve essere unica, mai vuota e stabile nel tempo. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Chiave esterna** (che la riferenzia da un'altra tabella) e **Database relazionale** (di cui è un elemento cardine).

Chunk

Un piccolo pezzo di testo (poche frasi o un paragrafo) in cui viene spezzato un documento lungo prima di elaborarlo con l'intelligenza artificiale, ad esempio nel RAG: dividere in pezzi più piccoli permette di recuperare solo la parte di documento davvero rilevante per una domanda, invece di doverlo leggere per intero ogni volta. → approfondito nella sezione 15 (Intelligenza artificiale).

CI (Continuous Integration)

La pratica di integrare frequentemente (più volte al giorno) il proprio codice con quello del resto del team, verificando automaticamente con dei test che tutto continui a funzionare insieme. → approfondito nella sezione 9 (DevOps) e nella sezione 10 (CI/CD). Si collega a: **Pipeline** (il meccanismo che la rende concreta) e **Quality Gate** (il controllo che ne verifica l'esito).

Ciclo di vita del software

L'insieme delle fasi che un software attraversa dalla sua nascita alla sua "pensione": analisi dei requisiti, progettazione, sviluppo, test, rilascio e manutenzione (software development lifecycle). → approfondito nella sezione 3 (Come nasce un software).

Client-Server

Un modello architetturale in cui un programma "client" (ad esempio il tuo browser) richiede informazioni o servizi a un programma "server", che risponde elaborando la richiesta. → approfondito nella sezione 11 (Architetture software).

Code Review

La pratica di far leggere e commentare il proprio codice a un collega prima di unirlo al progetto principale, per individuare errori, migliorare la qualità e condividere conoscenza nel team. → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Pull Request** (che la richiede) e **Merge** (l'operazione che la conclude, una volta approvata).

Commit

Uno “scatto fotografico” delle modifiche fatte al codice in un preciso momento, salvato nella storia del repository con un messaggio che ne descrive il contenuto. → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Branch** (su cui viene registrato) e **Pull Request** (che raccoglie una serie di commit da integrare).

Computer

Una macchina capace di eseguire istruzioni (programmi) per elaborare dati: è composta da componenti hardware (come CPU e RAM) che collaborano seguendo le indicazioni del software. → approfondito nella sezione 2 (Fondamenti di informatica).

Container

Un “pacchetto” leggero e isolato che contiene un’applicazione insieme a tutto ciò che le serve per funzionare (librerie, configurazioni), così da poterla eseguire in modo identico su qualsiasi computer. Docker è la tecnologia più usata per crearli. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Docker** (lo strumento più diffuso per crearli) e **Kubernetes** (che li orchestra su larga scala).

Continuous Delivery

La pratica che assicura che il software, dopo aver superato tutti i test automatici, sia sempre pronto per essere rilasciato in produzione in qualsiasi momento, anche se il rilascio finale resta una decisione manuale. → approfondito nella sezione 9 (DevOps). Si collega a: **CI (Continuous Integration)** (che la precede nel percorso verso il rilascio) e **Deploy/Rilascio** (l’operazione finale, qui ancora decisa manualmente).

Continuous Deployment

Un’evoluzione della Continuous Delivery in cui ogni modifica che supera i test viene rilasciata automaticamente in produzione, senza intervento manuale. → approfondito nella sezione 9 (DevOps). Si collega a: **Continuous Delivery** (di cui è l’evoluzione automatica) e **Deploy/Rilascio** (che qui avviene senza intervento umano).

Copilot (GitHub Copilot)

Uno strumento di AI generativa integrato nell’editor di codice che suggerisce automaticamente righe o intere funzioni mentre uno sviluppatore scrive, basandosi su un LLM addestrato su enormi quantità di codice pubblico. Non sostituisce lo sviluppatore: velocizza compiti ripetitivi, ma il codice suggerito va sempre letto e verificato in Code Review come qualsiasi altro contributo. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **AI generativa** (la categoria a cui appartiene) e **Code Review** (il controllo che il codice suggerito deve comunque superare).

CPI (Cost Performance Index)

Un indice dell'Earned Value Management che misura quanto efficientemente il progetto sta spendendo il proprio budget: si calcola dividendo il valore del lavoro effettivamente completato (EV) per il costo reale sostenuto (AC). Un CPI sopra 1 significa che il progetto sta spendendo meno di quanto pianificato per il lavoro fatto, sotto 1 significa che sta spendendo di più. → approfondito nella sezione 8 (Project Management). Si collega a: **EVM (Earned Value Management)** (il sistema di cui fa parte) e **SPI (Schedule Performance Index)** (l'indice analogo sui tempi).

CPU

Sigla di Central Processing Unit, il "cervello" del computer: il componente che esegue materialmente le istruzioni dei programmi, facendo calcoli a velocità elevatissima. → approfondito nella sezione 2 (Fondamenti di informatica).

Critical Path (percorso critico)

La sequenza di attività di un progetto che, messe una dopo l'altra, determinano la durata minima totale del progetto: se anche una sola di queste attività si ritarda, l'intero progetto si ritarda. Le attività non sul percorso critico hanno invece un margine di ritardo (la "slack") che non impatta la data finale. → approfondito nella sezione 8 (Project Management). Si collega a: **Diagramma di Gantt** (lo strumento visivo con cui si rappresenta) e **WBS (Work Breakdown Structure)** (da cui derivano le attività su cui si calcola).

CRUD

Acronimo di Create, Read, Update, Delete (Crea, Leggi, Aggiorna, Elimina): le quattro operazioni fondamentali che si possono fare su un qualsiasi dato salvato da qualche parte, corrispondenti ai verbi HTTP POST/GET/PUT-PATCH/DELETE e ai comandi SQL INSERT/SELECT/UPDATE/DELETE. Quando un tecnico dice "è solo un CRUD" indica una stima semplice, senza logica di business particolare. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **REST** (i verbi HTTP con cui si esprime) e **SQL** (i comandi con cui si realizza sul database).

Cultura DevOps

Un modo di lavorare che abbatte le barriere tra chi sviluppa il software (Dev) e chi lo gestisce in produzione (Ops), promuovendo collaborazione, responsabilità condivisa e automazione. → approfondito nella sezione 9 (DevOps).

Cycle Time

Il tempo che intercorre dal momento in cui il team **inizia effettivamente a lavorare** su un'attività al momento in cui la completa. È una misura più "interna" del Lead Time. → approfondito nella sezione 7 (Kanban).

Daily Scrum

Un breve incontro quotidiano (di solito 15 minuti) in cui il team Scrum sincronizza il lavoro, condivide progressi e segnala eventuali ostacoli. → approfondito nella sezione 6 (Scrum).

Data drift

Il progressivo “invecchiamento” di un modello di intelligenza artificiale che accade quando i dati reali su cui opera cambiano nel tempo rispetto a quelli usati durante il Training, facendo peggiorare silenziosamente le sue previsioni se nessuno se ne accorge. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (la base che diventa via via meno rappresentativa) e **MLOps** (la disciplina che si occupa di individuarlo e correggerlo).

Database NoSQL

Un database che non organizza i dati in tabelle rigide come quelli relazionali, ma in formati più flessibili (documenti, chiavi-valore, grafi), utile quando i dati cambiano forma spesso o servono grandi volumi. → approfondito nella sezione 2 (Fondamenti di informatica).

Database relazionale

Un database che organizza i dati in tabelle collegate tra loro tramite relazioni, un po' come fogli Excel che si richiamano a vicenda; si interroga con il linguaggio SQL. → approfondito nella sezione 2 (Fondamenti di informatica).

Database vettoriale

Un tipo particolare di database pensato per salvare e ricercare Embedding, cioè rappresentazioni numeriche del significato di un testo: invece di cercare parole esatte, permette di trovare rapidamente i contenuti più simili nel significato a una domanda, anche tra milioni di documenti. → approfondito nella sezione 15 (Intelligenza artificiale).

Debito tecnico

L'insieme dei compromessi tecnici presi per rilasciare più in fretta (codice scritto “alla svelta”, scorciatoie di progettazione, test saltati) che, come un debito finanziario, andranno prima o poi “ripagati” con lavoro di manutenzione, e che nel frattempo rendono più lento e rischioso ogni intervento successivo sul codice. → approfondito nella sezione 6 (Scrum).

Deep Learning

Un ramo del Machine Learning basato su reti neurali artificiali organizzate in molti “strati” (deep, cioè profondo), particolarmente efficace per compiti complessi come il riconoscimento di immagini o la comprensione del linguaggio naturale, ed è la tecnologia alla base della maggior parte degli LLM moderni. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Machine Learning** (il campo più ampio di cui è una tecnica specifica) e **LLM (Large Language Model)** (uno dei risultati più visibili di questa tecnica).

Definition of Done (DoD)

L'elenco condiviso di criteri che un'attività deve soddisfare per essere considerata davvero "completata" (ad esempio: codice scritto, testato, revisionato e documentato), evitando ambiguità nel team. → approfondito nella sezione 6 (Scrum).

Definition of Ready (DoR)

L'elenco di criteri che una User Story deve soddisfare prima di poter essere accettata in uno Sprint (ad esempio: requisiti chiari, stima fatta, dipendenze note). → approfondito nella sezione 6 (Scrum).

Deploy/Rilascio

L'operazione di rendere disponibile una nuova versione del software agli utenti, spostandola dall'ambiente in cui è stata sviluppata e testata a quello di produzione. → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Artifact (build)** (ciò che viene effettivamente distribuito) e **Rollback** (l'operazione da compiere se il rilascio va male).

Deriva di configurazione

Il progressivo disallineamento tra ambienti che dovrebbero somigliarsi (es. Test/QA, Staging, Produzione) quando una modifica di configurazione viene fatta in uno solo di essi e non replicata negli altri, portando col tempo a test che "passano" senza dire nulla sul comportamento reale. → approfondito nella sezione 14 (Ambienti di sviluppo).

Developer Experience (DX)

Quanto è facile, per chi scrive codice, capire cosa fare e farlo: documentazione chiara, messaggi d'errore utili, strumenti che non richiedono di leggere un manuale prima di poterli usare. È uno dei tre pilastri di un Internal Developer Platform, insieme a standardizzazione e automazione. → approfondito nella sezione 12 (Cloud). Si collega a: **IDP (Internal Developer Platform)** (lo strumento che la mette in pratica) e **Golden path** (il percorso raccomandato che la incarna).

DevOps

La fusione tra "Development" (sviluppo) e "Operations" (gestione dei sistemi): un approccio che unisce persone, processi e strumenti per rilasciare software in modo più rapido, frequente e affidabile. → approfondito nella sezione 9 (DevOps). Si collega a: **CALMS** (che ne riassume i pilastri) e **CI (Continuous Integration)** (una delle pratiche concrete che lo realizzano).

DevSecOps

Un'estensione del DevOps che integra la sicurezza (Security) in ogni fase del ciclo di vita del software, invece di considerarla solo un controllo finale prima del rilascio. → approfondito nella sezione 13 (Sicurezza).

Diagramma di Gantt

Un grafico a barre orizzontali che mostra le attività di un progetto lungo una linea del tempo, con inizio, fine e dipendenze reciproche visibili a colpo d'occhio. È lo strumento visivo classico della pianificazione “a cascata” (Waterfall), ma resta utile anche in un contesto Agile per comunicare scadenze e dipendenze a stakeholder che non seguono la board del team giorno per giorno. → approfondito nella sezione 8 (Project Management). Si collega a: **Critical Path** (il percorso che il diagramma rende visibile) e **WBS (Work Breakdown Structure)** (le attività che il diagramma colloca nel tempo).

Disco

Il componente del computer dove i dati vengono salvati in modo permanente (anche da spento), a differenza della RAM che perde i dati quando il computer si spegne. → approfondito nella sezione 2 (Fondamenti di informatica).

DNS

Sigla di Domain Name System: il “elenco telefonico” di internet, che traduce nomi di siti facili da ricordare (come esempio.it) negli indirizzi numerici (IP) che i computer usano davvero per comunicare. → approfondito nella sezione 2 (Fondamenti di informatica).

Docker

Lo strumento più diffuso per creare, eseguire e distribuire Container, cioè applicazioni “pacchettizzate” insieme a tutto ciò che serve loro per funzionare ovunque nello stesso modo. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Container** (ciò che crea e distribuisce) e **Artifact (build)** (un'immagine Docker è uno dei formati più comuni di artifact).

Embedding

Una rappresentazione numerica (una lista di numeri) del significato di un testo, un'immagine o altro contenuto, calcolata da un modello di intelligenza artificiale: due contenuti con significato simile avranno un embedding simile, anche se usano parole diverse. È la base tecnica che rende possibile la Ricerca semantica. → approfondito nella sezione 15 (Intelligenza artificiale).

Epic

Un'attività molto grande, che raggruppa più User Story legate a un unico obiettivo di ampio respiro, troppo estesa per essere completata in un singolo Sprint. → approfondito nella sezione 6 (Scrum).

Event-Driven Architecture

Un'architettura in cui un servizio, invece di chiamare direttamente chi deve fare qualcosa, pubblica un evento (un fatto accaduto) su un topic dedicato; chi è interessato (consumer) reagisce in autonomia, senza che chi pubblica (producer) debba conoscerlo. Il meccanismo con cui producer e consumer si incontrano si chiama pub/sub. → approfondito nella sezione 11 (Architetture software). Si collega a: **Evento** (l'unità che vi circola) e **Message Queue** (l'infrastruttura sottostante, usata qui in modo diverso).

Evento

Un fatto accaduto al passato, per definizione immutabile (es. “la polizza è stata sottoscritta”): a differenza di un comando, non è indirizzato a nessuno in particolare, non richiede una risposta e non può essere “rifiutato” perché è già successo. → approfondito nella sezione 11 (Architetture software). Si collega a: **Event-Driven Architecture** (il pattern che si basa su di esso).

EVM (Earned Value Management)

Una tecnica di Project Management che confronta tre valori — quanto lavoro era pianificato (PV, Planned Value), quanto è stato effettivamente completato in termini di valore (EV, Earned Value) e quanto è realmente costato (AC, Actual Cost) — per capire in modo oggettivo se un progetto è in linea con tempi e budget, invece di basarsi solo su una sensazione. → approfondito nella sezione 8 (Project Management). Si collega a: **CPI (Cost Performance Index)** e **SPI (Schedule Performance Index)** (i due indici che questa tecnica calcola).

Feature

Una funzionalità del software: una caratteristica o capacità concreta che l'utente può usare (ad esempio “possibilità di esportare un report in PDF”). → approfondito nella sezione 3 (Come nasce un software).

Feature flag

Un interruttore nel codice che permette di attivare o disattivare una funzionalità già distribuita in produzione senza fare un nuovo rilascio, ad esempio per mostrarla solo a un gruppo ristretto di utenti o per disattivarla rapidamente se causa problemi. → approfondito nella sezione 10 (CI/CD).

File System

Il modo in cui un sistema operativo organizza e conserva i file su un disco, tramite cartelle e nomi, permettendo di ritrovarli e gestirli facilmente. → approfondito nella sezione 2 (Fondamenti di informatica).

Fine-tuning

Il processo di riaddestrare ulteriormente un modello di intelligenza artificiale già pronto, usando dati specifici di un'azienda o di un compito, per specializzarlo su quel contesto invece di usarlo “come viene”. È un'alternativa al RAG per dare a un modello conoscenza specifica: più lenta e costosa da aggiornare, ma capace di cambiare più a fondo il comportamento del modello. → approfondito nella sezione 15 (Intelligenza artificiale).

FinOps

La disciplina (dall'unione di “Finance” e “Operations”) che si occupa di gestire, prevedere e ottimizzare la spesa cloud di un'azienda, tramite allarmi di spesa, attribuzione dei costi ai progetti giusti e collaborazione tra team tecnici e finanziari. → approfondito nella sezione 12 (Cloud).

Frontend

La parte di un'applicazione con cui l'utente interagisce direttamente: l'interfaccia visibile (pagine web, schermate, pulsanti), che comunica con il Backend per mostrare e inviare informazioni. → approfondito nella sezione 11 (Architetture software).

GDPR

Il Regolamento Generale sulla Protezione dei Dati (General Data Protection Regulation), la legge europea che stabilisce come le aziende devono raccogliere, trattare e proteggere i dati personali delle persone. → approfondito nella sezione 13 (Sicurezza).

Git Flow

Un modello ben definito di organizzazione dei Branch in Git, con rami dedicati per lo sviluppo, le funzionalità, i rilasci e le correzioni urgenti, utile in progetti con rilasci pianificati e meno frequenti. → approfondito nella sezione 4 (Git e GitHub).

Golden path

Il percorso raccomandato e già pronto per costruire e rilasciare un servizio, che copre la maggior parte dei casi tipici: non l'unico percorso possibile (un team con esigenze particolari può uscirne), ma quello che rende semplice fare la cosa giusta senza dover reinventare tutto da zero. → approfondito nella sezione 12 (Cloud). Si collega a: **IDP (Internal Developer Platform)** (lo strumento che lo rende disponibile) e **Self-service** (il modo in cui viene ottenuto).

HTML

Il linguaggio di marcatura che descrive la struttura di una pagina web (titoli, paragrafi, link, immagini) tramite tag, dicendo al browser cosa mostrare; a differenza di Markdown, gestisce anche la formattazione visiva più fine (tramite CSS) ed è il "compilato" verso cui Markdown viene spesso convertito. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Markdown** (il linguaggio più semplice da cui viene spesso generato) e **XML** (un altro linguaggio a tag, ma pensato per i dati non per le pagine).

HTTP

Sigla di HyperText Transfer Protocol: il linguaggio con cui il browser e i siti web si scambiano informazioni su internet, ad esempio quando richiedi una pagina web. → approfondito nella sezione 2 (Fondamenti di informatica).

HTTPS

La versione sicura e cifrata di HTTP (la "S" sta per Secure), che protegge le informazioni scambiate tra browser e sito web da occhi indiscreti, oggi usata praticamente ovunque. → approfondito nella sezione 2 (Fondamenti di informatica).

IaaS

Sigla di Infrastructure as a Service: un modello cloud in cui il fornitore ti dà “solo” server, storage e rete virtuali, e sei tu a occuparti di sistema operativo, applicazioni e configurazioni. → approfondito nella sezione 12 (Cloud).

IaC (Infrastructure as Code)

La pratica di descrivere l'infrastruttura informatica (server, reti, configurazioni) tramite file di testo/codice invece che configurandola manualmente, così da poterla creare, versionare e ripetere in modo automatico e affidabile. → approfondito nella sezione 9 (DevOps).

Idempotenza

La proprietà di un'operazione per cui eseguirla più volte con lo stesso input produce lo stesso risultato di eseguirla una sola volta: fondamentale per i consumer di un'architettura event-driven, dove un evento può arrivare duplicato (es. controllare “ho già incassato questo premio?” invece di incassarlo incondizionatamente). → approfondito nella sezione 11 (Architetture software). Si collega a: **Event-Driven Architecture** (il contesto in cui diventa cruciale) e **Evento** (che può arrivare più di una volta allo stesso consumer).

IDP (Internal Developer Platform)

Il livello di self-service che un team di piattaforma costruisce sopra l'infrastruttura (cloud, Kubernetes, pipeline), così che un team di sviluppo possa ottenere ambienti, database o pipeline senza passare ogni volta da zero. Si basa su tre pilastri: standardizzazione, automazione e Developer Experience. → approfondito nella sezione 12 (Cloud). Si collega a: **Platform Engineering** (la disciplina che lo costruisce) e **Golden path** (il percorso che offre).

Increment

Il risultato concreto e utilizzabile prodotto durante uno Sprint: la somma di tutte le User Story completate, che si aggiunge a quanto già costruito nei Sprint precedenti. → approfondito nella sezione 6 (Scrum).

Inferenza

Il momento in cui un modello di intelligenza artificiale già addestrato viene effettivamente usato per produrre una risposta o una previsione su un nuovo dato in ingresso, distinto dalla fase di Training in cui il modello impara. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (la fase precedente, in cui il modello apprende) e **Modello (AI)** (l'oggetto che, una volta addestrato, viene interrogato in questa fase).

Issue

Una “segnalazione” aperta in un repository (su GitHub Issues) per tracciare un bug, una richiesta di funzionalità o qualsiasi attività da discutere e risolvere. → approfondito nella sezione 4 (Git e GitHub).

ITIL

Il framework di riferimento più diffuso per l'ITSM (attualmente alla versione ITIL 4): una raccolta di buone pratiche, non uno standard obbligatorio, usata da moltissime organizzazioni enterprise come vocabolario comune per organizzare i processi di gestione dei servizi IT. → approfondito nella sezione 9 (DevOps). Si collega a: **ITSM** (la disciplina che descrive).

ITSM (IT Service Management)

L'insieme di processi con cui un'organizzazione IT gestisce i servizi che fornisce, perché funzionino in modo prevedibile tutti i giorni, non solo il giorno del rilascio: incident management (ripristinare il servizio), problem management (trovare la causa radice), change management (autorizzare le modifiche) e service request (richieste che non sono guasti). → approfondito nella sezione 9 (DevOps). Si collega a: **ITIL** (il framework di riferimento) e **Change Request (richiesta di modifica)** (l'equivalente PMP del concetto di change management).

JOIN

L'istruzione SQL che unisce temporaneamente le righe di due tabelle collegate tra loro tramite una chiave esterna, per recuperare insieme informazioni che vivono in tabelle diverse. Una INNER JOIN restituisce solo le righe con corrispondenza in entrambe le tabelle; una LEFT JOIN restituisce anche quelle senza corrispondenza. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Chiave esterna** (su cui si basa) e **Normalizzazione** (il principio che rende la JOIN necessaria).

JQL (Jira Query Language)

Il linguaggio di interrogazione di Jira che permette di costruire ricerche precise sulle issue (per progetto, assegnatario, stato, data) invece di filtrare a mano una lista lunghissima, ed è alla base della maggior parte dei report e delle dashboard costruite su Jira. → approfondito nella sezione 8 (Project Management).

JSON

Sigla di JavaScript Object Notation: un formato di testo semplice e leggibile usato moltissimo per scambiare dati tra programmi, ad esempio nelle risposte delle API. → approfondito nella sezione 2 (Fondamenti di informatica).

JWT (JSON Web Token)

Un token compatto e firmato, composto da tre parti (header.payload.signature), usato per l'autenticazione stateless: contiene già i dati sull'identità di chi lo possiede, così il server può verificarne l'autenticità senza dover consultare nulla, a differenza di una sessione server-side. Va usato con una scadenza breve, perché non può essere revocato a metà strada come una sessione. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Token** (di cui è un'implementazione concreta molto diffusa) e **OAuth** (che spesso lo usa come formato dell'access token).

Knowledge Area (area di conoscenza)

Nell'impostazione del PMBOK Guide 6ª edizione, una delle 10 categorie in cui viene organizzata la conoscenza necessaria a gestire un progetto (ad esempio i costi, i tempi, la qualità, le comunicazioni). Ogni area raggruppa i processi legati a quel tema specifico, indipendentemente dalla fase del progetto in cui si applicano. → approfondito nella sezione 8 (Project Management). Si collega a: **Process Group (gruppo di processi)** (l'altra dimensione, complementare, con cui il PMBOK 6 organizza gli stessi processi).

KPI

Sigla di Key Performance Indicator: un indicatore numerico che misura quanto bene un progetto o un'attività sta raggiungendo i propri obiettivi (ad esempio il tempo medio di risposta a un cliente). → approfondito nella sezione 8 (Project Management).

Kubernetes

Uno strumento che gestisce automaticamente grandi quantità di Container in produzione: li avvia, li riavvia se si bloccano, li distribuisce su più macchine e ne regola il numero in base al carico. → approfondito nella sezione 2 (Fondamenti di informatica).

LDAP (Lightweight Directory Access Protocol)

Il protocollo con cui le applicazioni interrogano una directory di identità (tipicamente Active Directory) per verificare utenti, gruppi e permessi, senza che ogni applicazione debba reinventare la propria gestione degli accessi. → approfondito nella sezione 13 (Sicurezza). Si collega a: **Active Directory (AD)** (l'implementazione più diffusa che lo parla).

Lead Time

Il tempo totale che intercorre dal momento in cui un'attività viene **richiesta** al momento in cui viene **consegnata**, includendo anche l'attesa prima che il lavoro inizi davvero. → approfondito nella sezione 7 (Kanban).

Legge di Conway

L'osservazione secondo cui la struttura di un software tende a rispecchiare la struttura del team che lo costruisce, e viceversa: scomporre un'applicazione in servizi per dominio porta spesso, prima o poi, a riorganizzare anche le persone in squadre allineate a quegli stessi domini. → approfondito nella sezione 11 (Architetture software).

Lessons Learned

La raccolta strutturata, alla chiusura di un progetto (o di una sua fase), di cosa ha funzionato bene e cosa no, così da non ripetere gli stessi errori nei progetti successivi. → approfondito nella sezione 8 (Project Management). Si collega a: **Sprint Retrospective** (l'equivalente Agile, ma ripetuto a ogni Sprint invece che solo a fine progetto).

Livello di isolamento

Una delle configurazioni (Read Uncommitted, Read Committed, Repeatable Read, Serializable) con cui un database stabilisce quanto rigorosamente proteggere una transazione dagli effetti di altre transazioni concorrenti: livelli più rigorosi prevengono più problemi (dirty read, non-repeatable read, phantom read) ma costano in performance. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **ACID** (la proprietà Isolation che questi livelli implementano in pratica) e **MVCC** (la tecnica con cui molti database moderni realizzano l'isolamento senza bloccare le letture).

LLM (Large Language Model)

Sigla di Large Language Model, un modello di intelligenza artificiale addestrato su enormi quantità di testo per prevedere e generare linguaggio naturale in modo fluido; è la tecnologia dietro strumenti come

ChatGPT o GitHub Copilot. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **AI generativa** (la categoria di utilizzo più diffusa per questi modelli) e **Token** (l'unità in cui il modello scompone il testo per elaborarlo).

Load Balancer (bilanciatore di carico)

Un componente che riceve tutte le richieste in arrivo verso un sistema e le distribuisce su più server applicativi identici, invece di farle gestire a un unico server: se un server si guasta il traffico continua a fluire sugli altri, e se il traffico cresce basta aggiungere un altro server dietro lo stesso bilanciatore. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Scalabilità verticale e orizzontale** (lo strumento che rende possibile la scalabilità orizzontale) e **Client-Server** (il modello che, per funzionare con più server dietro un load balancer, richiede che le richieste non dipendano da uno stato salvato su un server specifico).

Lock-in

La dipendenza da un provider cloud (o da un fornitore di software) che si crea usando i suoi servizi specifici: più se ne usano, più diventa costoso, in tempo e denaro, cambiare fornitore in futuro. Non è un divieto ad usare quei servizi, ma un costo da valutare consapevolmente. → approfondito nella sezione 12 (Cloud).

Logging

La pratica di registrare in modo continuo gli eventi che accadono in un sistema (errori, richieste, azioni) in file o strumenti dedicati, utile per capire cosa è successo quando qualcosa va storto. → approfondito nella sezione 9 (DevOps).

Lost Update (aggiornamento perso)

Un problema di concorrenza in cui due transazioni leggono lo stesso valore, lo modificano entrambe e l'ultima sovrascrive silenziosamente il lavoro della prima, senza nessun errore visibile: ad esempio due operatori che aggiornano lo stock dello stesso prodotto contemporaneamente. → approfondito nella

sezione 20 (Backend Engineering Avanzato). Si collega a: **Livello di isolamento** (il meccanismo che, se scelto correttamente, previene questo problema) e **Race Condition** (di cui è un caso specifico, applicato a una transazione di database).

Machine Learning

Un ramo dell'intelligenza artificiale in cui un sistema impara a svolgere un compito analizzando esempi (dati) invece di seguire regole scritte a mano da un programmatore, migliorando le proprie previsioni con l'esperienza. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **AI (Intelligenza Artificiale)** (il campo più ampio di cui è una branca) e **Deep Learning** (una delle sue tecniche più diffuse oggi).

Manifesto Agile

Il documento pubblicato nel 2001 da un gruppo di sviluppatori che definisce i valori e i principi alla base dell'approccio Agile, come privilegiare le persone e la collaborazione rispetto a processi rigidi e documentazione eccessiva. → approfondito nella sezione 5 (Agile).

Markdown

Un linguaggio di marcatura leggero per scrivere documenti formattati (titoli, elenchi, grassetto, link) usando pochi simboli semplici (`#`, `*`, `[]()`), pensato per essere leggibile anche senza essere convertito, a differenza dell'HTML. Questo stesso corso è scritto in Markdown. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **HTML** (il linguaggio più ricco verso cui viene spesso convertito).

Maven

Uno strumento di build e gestione delle dipendenze per progetti Java: legge un file di configurazione (il POM) che dichiara le librerie esterne necessarie e le scarica automaticamente, insieme alle loro dipendenze transitive, invece di doverle procurare e aggiornare a mano una per una. → approfondito nella sezione 4 (Git e GitHub). Si collega a: **POM** (il file che lo configura).

MCP (Model Context Protocol)

Sigla di Model Context Protocol: uno standard aperto, introdotto da Anthropic e con adozione crescente, che definisce un modo comune con cui un assistente AI si collega a fonti di dati e strumenti esterni, un po' come una presa USB-C che permette di collegare dispositivi diversi con lo stesso connettore. Prima di MCP, ogni assistente andava collegato "a mano" a ogni sistema con cui doveva interagire; MCP evita questa moltiplicazione di integrazioni su misura. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **LLM (Large Language Model)** (l'assistente che, tramite MCP, si collega agli strumenti esterni) e **API** (il modo con cui, dietro le quinte, un MCP server spesso espone le proprie capacità).

MCP server

Il componente che, seguendo lo standard MCP, espone a un assistente AI le capacità di un sistema — ad esempio leggere dei file, interrogare un database o chiamare un'API — così che l'assistente possa scegliere quando e come usarle per rispondere a una richiesta. → approfondito nella sezione 15 (Intelligenza artificiale).

Merge

L'operazione con cui le modifiche fatte su un Branch vengono riunite (unite) al codice principale o a un altro branch. → approfondito nella sezione 3 (Come nasce un software).

Message Queue

Un sistema che permette a diverse parti di un'applicazione di scambiarsi messaggi senza dover comunicare direttamente e nello stesso istante: un componente “deposita” un messaggio in una coda e un altro lo legge quando è pronto. → approfondito nella sezione 11 (Architetture software).

MFA (autenticazione a più fattori)

Un metodo di sicurezza che richiede più di una prova d'identità per accedere a un sistema (ad esempio password più codice ricevuto sul telefono), rendendo molto più difficile un accesso non autorizzato anche se la password viene scoperta. → approfondito nella sezione 13 (Sicurezza).

Microservizi

Un'architettura software in cui un'applicazione è divisa in tanti piccoli servizi indipendenti, ciascuno responsabile di una funzione specifica, che comunicano tra loro (spesso via API), invece di essere un unico grande blocco di codice. → approfondito nella sezione 11 (Architetture software).

Milestone

Un punto di controllo significativo in un progetto, che segna il completamento di una fase importante o il raggiungimento di un obiettivo intermedio. → approfondito nella sezione 8 (Project Management).

Mindset Agile

L'atteggiamento mentale alla base di Agile: adattarsi al cambiamento, imparare dagli errori, collaborare con il cliente e migliorare continuamente, più che l'applicazione meccanica di regole e strumenti. → approfondito nella sezione 5 (Agile).

MLOps

L'insieme di pratiche che estendono i principi del DevOps (automazione, collaborazione, versionamento) al ciclo di vita dei modelli di Machine Learning: dalla preparazione dei dati al Training, dal deploy in produzione al monitoraggio delle prestazioni nel tempo. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **DevOps** (di cui applica i principi al mondo dei modelli AI) e **Data drift** (uno dei problemi che il monitoraggio MLOps serve a individuare).

Modello (AI)

Il risultato concreto del Training: una sorta di “funzione matematica” che ha imparato a riconoscere pattern nei dati e che, una volta pronta, viene usata in fase di Inferenza per produrre risposte o previsioni su nuovi dati. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (il processo che lo produce) e **Inferenza** (il momento in cui viene effettivamente usato).

Monitoring

L'attività di osservare costantemente lo stato di un sistema (uso della CPU, tempi di risposta, errori) per accorgersi rapidamente se qualcosa non funziona come dovrebbe. → approfondito nella sezione 9 (DevOps).

Monolite

Un'architettura software in cui tutta l'applicazione è costruita come un unico grande blocco di codice, con tutte le funzionalità interdipendenti tra loro, contrapposta ai Microservizi. → approfondito nella sezione 11 (Architetture software).

Multi-stage build

Una tecnica per scrivere un Dockerfile in più fasi (stage): una “builder image” compila l'applicazione con tutti gli strumenti di sviluppo necessari, poi solo il risultato della compilazione viene copiato in un'immagine finale più piccola e senza quegli strumenti, che è quella che finisce davvero in produzione. → approfondito nella sezione 10 (CI/CD). Si collega a: **Builder image** (la fase intermedia che rende possibile) e **Artifact (build)** (il prodotto finale, più leggero, che ne risulta).

MVCC (Multi-Version Concurrency Control)

Una tecnica usata dai database moderni (PostgreSQL, MySQL InnoDB) per gestire l'isolamento tra transazioni senza bloccare tutto: quando una riga viene modificata, il database mantiene più versioni della stessa riga contemporaneamente, così chi legge continua a vedere una versione coerente mentre chi scrive lavora su una nuova versione — i lettori non bloccano gli scrittori, e viceversa. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Livello di isolamento** (il comportamento che questa tecnica realizza in pratica) e **ACID** (la proprietà Isolation di cui è un'implementazione efficiente).

Normalizzazione

Il principio secondo cui ogni informazione in un database relazionale viene scritta una volta sola, nel posto giusto, e recuperata altrove tramite relazioni e JOIN quando serve, invece di essere ripetuta in più righe (con il rischio che, se cambia, venga aggiornata in un posto e dimenticata in un altro). → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **JOIN** (lo strumento che rende possibile recuperare i dati normalizzati) e **Chiave esterna** (su cui si basano le relazioni normalizzate).

OAuth

Un protocollo che permette di delegare un'autorizzazione limitata e revocabile a un'applicazione terza, senza mai cederle la password (es. "puoi leggere solo i miei documenti, per i prossimi 30 giorni"): è il meccanismo dietro al pulsante "accedi con..." di tanti siti. → approfondito nella sezione 13 (Sicurezza). Si collega a: **OpenID Connect** (che aggiunge l'autenticazione allo stesso meccanismo) e **Token** (lo strumento con cui l'autorizzazione viene concessa in pratica).

Observability

La capacità di capire **cosa sta succedendo dentro** un sistema complesso osservandolo dall'esterno, combinando log, metriche e tracce; va oltre il semplice Monitoring perché aiuta a capire anche il "perché" di un problema, non solo il "cosa". → approfondito nella sezione 9 (DevOps).

OpenAPI / Swagger

OpenAPI è uno standard per descrivere un'API REST (risorse, verbi, campi di richiesta e risposta) in un formato leggibile sia da una persona che da una macchina, trasformandola da "documento di cui ci si fida" a contratto verificabile tra due team. Swagger è il nome della famiglia di strumenti storicamente associata a OpenAPI, che genera documentazione navigabile e codice di base a partire dalla stessa descrizione. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **REST** (lo stile di API che descrive) e **JSON** (uno dei formati in cui la specifica è tipicamente scritta).

OpenID Connect (OIDC)

Un livello costruito sopra OAuth che aggiunge l'autenticazione vera e propria ("dimmi chi è questa persona"), non solo la delega di un permesso limitato: OAuth risponde a "cosa posso fare per conto tuo", OIDC risponde anche a "chi sono". → approfondito nella sezione 13 (Sicurezza). Si collega a: **OAuth** (il protocollo su cui si basa) e **SSO (Single Sign-On)** (uno degli usi pratici più comuni di questa combinazione).

OpenShift (OCP)

Il prodotto di Red Hat che racchiude Kubernetes con registry, autenticazione, rete, pipeline e monitoraggio già integrati, oltre a un supporto commerciale e a vincoli di sicurezza più severi per impostazione predefinita (es. niente container con privilegi di root). OpenShift è Kubernetes, non lo sostituisce. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Kubernetes** (che racchiude) e **Platform Engineering** (di cui è un mattone possibile, non l'intera piattaforma).

Overfitting

Un problema che si verifica quando un modello di intelligenza artificiale "impara a memoria" i dati di Training invece di generalizzare, risultando molto bravo sugli esempi già visti ma poco affidabile su dati nuovi. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (la fase in cui si verifica) e **Modello (AI)** (che ne risulta compromesso, con previsioni poco affidabili su casi nuovi).

OWASP

Sigla di Open Web Application Security Project: un'organizzazione che pubblica risorse gratuite (come la celebre lista "OWASP Top 10") sulle vulnerabilità di sicurezza più comuni nelle applicazioni web. → approfondito nella sezione 13 (Sicurezza).

PaaS

Sigla di Platform as a Service: un modello cloud in cui il fornitore gestisce anche il sistema operativo e gli strumenti di base, lasciandoti concentrare solo sullo sviluppo e sulla gestione della tua applicazione. → approfondito nella sezione 12 (Cloud).

Pay-as-you-go

Un modello di costo cloud in cui paghi solo per le risorse effettivamente usate (come una bolletta elettrica), invece di acquistare in anticipo hardware costoso che magari resterà sottoutilizzato. → approfondito nella sezione 12 (Cloud).

Pipeline

Una sequenza automatizzata di passaggi (compilare, testare, rilasciare) che il codice attraversa dal momento in cui viene scritto a quando arriva agli utenti, senza interventi manuali ripetitivi. → approfondito nella sezione 10 (CI/CD). Si collega a: **Trigger** (l'evento che ne fa scattare l'avvio) e **Artifact (build)** (il prodotto finale che genera).

Pipeline as Code

La pratica di definire i passaggi di una pipeline in un file di testo versionato insieme al codice, invece che configurarla a mano tramite un'interfaccia grafica, così da poterla tracciare e riutilizzare facilmente. → approfondito nella sezione 10 (CI/CD).

Platform Engineering

La disciplina di costruire e mantenere una piattaforma interna per gli sviluppatori, trattata come un prodotto con dei clienti (i team di sviluppo) che possono, in linea di principio, scegliere di non usarla — a differenza di un ufficio che impone strumenti per decreto. Nasce come risposta al debito organizzativo lasciato scoperto dal DevOps preso alla lettera. → approfondito nella sezione 12 (Cloud). Si collega a: **IDP (Internal Developer Platform)** (ciò che concretamente costruisce) e **DevOps** (la cultura di cui completa la promessa).

PMBOK Guide

Sigla di Project Management Body of Knowledge: la guida pubblicata dal PMI che raccoglie le conoscenze, i processi e le pratiche di riferimento del Project Management. Attenzione: le edizioni non sono tutte uguali — la 6ª edizione è basata su 5 gruppi di processi e 10 aree di conoscenza, mentre la 7ª edizione (2021) ha cambiato impostazione, basandosi su 12 principi e 8 performance domain. Quando qualcuno cita "il PMBOK" senza specificare l'edizione, vale la pena chiedere a quale delle due

si riferisce. → approfondito nella sezione 8 (Project Management). Si collega a: **PMI (Project Management Institute)** (l'organizzazione che la pubblica) e **PMP (Project Management Professional)** (la certificazione che su questa guida si basa).

PMI (Project Management Institute)

L'organizzazione internazionale di riferimento per il Project Management: pubblica il PMBOK Guide, rilascia certificazioni professionali (come il PMP) e promuove standard e buone pratiche condivise nel settore. → approfondito nella sezione 8 (Project Management).

PMP (Project Management Professional)

Una certificazione professionale rilasciata dal PMI, tra le più riconosciute al mondo per chi gestisce progetti: attesta la conoscenza dei processi e delle pratiche descritte nel PMBOK Guide e un certo numero di anni di esperienza sul campo. Per i requisiti aggiornati (esperienza richiesta, costi, modalità d'esame) fai sempre riferimento al sito ufficiale del PMI, perché cambiano nel tempo. → approfondito nella sezione 8 (Project Management).

POM (Project Object Model)

Il file (tipicamente `pom.xml`) con cui Maven configura la build di un progetto Java: dichiara nome e versione del progetto, le dipendenze esterne necessarie con la loro versione precisa, e le fasi della build (compile, test, package). → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Maven** (lo strumento che lo legge ed esegue).

Process Group (gruppo di processi)

Nell'impostazione del PMBOK Guide 6ª edizione, una delle 5 macro-fasi in cui si organizzano i processi di un progetto: Avvio, Pianificazione, Esecuzione, Monitoraggio e Controllo, Chiusura. Non sono fasi rigidamente sequenziali quanto categorie di attività che possono anche sovrapporsi nel tempo. → approfondito nella sezione 8 (Project Management). Si collega a: **Knowledge Area (area di conoscenza)** (l'altra dimensione con cui il PMBOK 6 organizza gli stessi processi).

Processo

Un programma in esecuzione sul computer, con la propria area di memoria dedicata: quando apri un'applicazione, il sistema operativo crea un processo per farla funzionare. → approfondito nella sezione 2 (Fondamenti di informatica).

Procurement (approvvigionamenti)

L'attività di project management che riguarda l'acquisto di beni o servizi esterni necessari al progetto (ad esempio un fornitore, una licenza software, una consulenza specialistica), inclusa la scelta del fornitore e la gestione del contratto. → approfondito nella sezione 8 (Project Management).

Product Backlog

L'elenco ordinato di tutto ciò che potrebbe essere fatto su un prodotto: funzionalità, correzioni, miglioramenti. È gestito dal Product Owner e cambia continuamente in base alle priorità. → approfondito nella sezione 6 (Scrum). Si collega a: **Product Owner** (che lo gestisce) e **Sprint Backlog** (il sottoinsieme selezionato per lo sprint corrente).

Product Owner

La persona responsabile di definire cosa deve essere costruito e in quale ordine di priorità, rappresentando la voce del cliente e degli utenti all'interno del team Scrum. → approfondito nella sezione 6 (Scrum).

Produzione

L'ambiente "vero", quello effettivamente usato dagli utenti finali del software; è l'ultimo gradino dopo Sviluppo, Test/QA e Staging, e qui gli errori hanno un impatto reale. → approfondito nella sezione 14 (Ambienti di sviluppo).

Project Charter

Il documento che autorizza formalmente l'esistenza di un progetto: definisce obiettivi, sponsor, vincoli principali e dà al project manager l'autorità per usare le risorse necessarie. È tipicamente uno dei primi documenti prodotti, nel gruppo di processi di Avvio. → approfondito nella sezione 8 (Project Management). Si collega a: **Sponsor** (chi lo firma e autorizza) e **Product Backlog** (l'equivalente Agile più operativo, che nasce dopo, una volta che il progetto è già autorizzato).

Prompt

Il testo (un'istruzione, una domanda, una richiesta) che un utente fornisce a un modello di AI generativa per ottenere una risposta: la qualità del prompt influenza molto la qualità della risposta ottenuta. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **AI generativa** (il tipo di strumento con cui si usa) e **LLM (Large Language Model)** (il modello che lo interpreta per generare una risposta).

Pub/Sub (publish/subscribe)

Il meccanismo con cui producer e consumer si incontrano in un'architettura event-driven: il producer pubblica un evento su un canale dedicato (topic), e ogni consumer interessato si sottoscrive a quel topic per riceverne una copia, tramite un broker che fa da intermediario. → approfondito nella sezione 11 (Architetture software). Si collega a: **Event-Driven Architecture** (il pattern che lo usa) e **Message Queue** (l'infrastruttura sottostante, usata qui in modo diverso: non un messaggio per un destinatario noto, ma un evento per un numero qualsiasi di sottoscrittori).

Pull Request

Una richiesta formale di unire le proprie modifiche di codice (spesso su un Branch) al codice principale, che di solito viene prima discussa e revisionata dal team (Code Review) prima di essere accettata: è il termine usato da GitHub per questo meccanismo (spesso abbreviata PR). → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Branch** (che la genera) e **Code Review** (che ne consegue, prima dell'approvazione).

Quality Gate

Un controllo automatico inserito in una pipeline che blocca l'avanzamento del rilascio se certi criteri di qualità non sono soddisfatti (ad esempio troppi errori nei test o problemi di sicurezza rilevati). → approfondito nella sezione 10 (CI/CD). Si collega a: **CI (Continuous Integration)** (di cui è parte integrante) e **Artifact (build)** (che viene generato solo se il gate viene superato).

RACI

Una matrice usata in Project Management per chiarire i ruoli in un'attività: chi è Responsible (la esegue), chi è Accountable (ne risponde), chi va Consultato e chi va solo Informato. → approfondito nella sezione 8 (Project Management).

RAG (Retrieval-Augmented Generation)

Sigla di Retrieval-Augmented Generation, cioè “generazione aumentata dal recupero”: una tecnica che, invece di riaddestrare un modello di intelligenza artificiale, gli fornisce al momento della domanda i documenti più rilevanti recuperati da una base documentale aziendale, così che possa rispondere basandosi su fonti verificabili invece che solo su ciò che ha imparato durante il Training. Ha però dei limiti: se recupera il documento sbagliato la risposta sarà sbagliata, e la qualità dipende sempre da quella della documentazione di partenza. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Allucinazione (AI)** (il rischio che questa tecnica riduce, senza eliminarlo del tutto) e **Fine-tuning** (l'alternativa, più costosa da aggiornare, di riaddestrare il modello invece di fornirgli i documenti al momento della domanda).

RAID Log

Un registro che il project manager tiene per tracciare Risks (rischi), Assumptions (assunzioni), Issues (problemi) e Dependencies (dipendenze) di un progetto, per avere sempre sotto controllo i punti critici. → approfondito nella sezione 8 (Project Management).

Race Condition

Un problema che si verifica quando più thread (o processi) accedono e modificano lo stesso dato condiviso contemporaneamente senza coordinamento, producendo un risultato finale inconsistente e dipendente dall'ordine casuale con cui le operazioni si sono sovrapposte. Si previene con lock (mutex) o strutture dati pensate apposta per la concorrenza. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Thread** (l'unità di esecuzione tra cui il problema si manifesta) e **Lost Update** (un caso specifico di race condition applicato a una transazione di database).

RAM

Sigla di Random Access Memory: la memoria “di lavoro” del computer, veloce ma temporanea, che perde i dati quando il computer viene spento — a differenza del Disco. → approfondito nella sezione 2 (Fondamenti di informatica).

Release

Una versione del software resa disponibile agli utenti, spesso accompagnata da un elenco delle novità e delle correzioni incluse rispetto alla versione precedente. → approfondito nella sezione 4 (Git e GitHub).

Replication

La strategia di scaling che mantiene più copie identiche dello stesso database: un nodo primario (master) gestisce tutte le scritture e le propaga a uno o più nodi replica (slave) dedicati alle letture, distribuendo così il carico di lettura e fornendo alta disponibilità se il primario si guasta. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Sharding** (la strategia complementare, che scala i dati invece delle sole letture) e **Scalabilità verticale e orizzontale** (di cui è una forma pratica di scalabilità orizzontale).

Repository

Lo “spazio” (cartella speciale) dove Git conserva tutto il codice di un progetto insieme alla sua intera storia di modifiche (Commit). → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Commit** (che ne costituisce la storia) e **Branch** (che ne organizza lo sviluppo in parallelo).

Requisiti (funzionali e non funzionali)

Le caratteristiche che un software deve avere: i requisiti funzionali descrivono **cosa** il sistema deve fare (es. “l’utente può resettare la password”), quelli non funzionali descrivono **come** deve farlo (es. velocità, sicurezza, disponibilità). → approfondito nella sezione 3 (Come nasce un software).

REST

Uno stile molto diffuso per progettare API web, basato su regole semplici e standard (come l’uso di HTTP) che rendono i servizi facili da capire, usare e collegare tra loro. → approfondito nella sezione 2 (Fondamenti di informatica).

Rete

Un insieme di computer e dispositivi collegati tra loro per scambiarsi dati e comunicare, dalla piccola rete di un ufficio fino a internet, la rete di reti più grande al mondo. → approfondito nella sezione 2 (Fondamenti di informatica).

Ricerca semantica

Una ricerca che trova i contenuti più simili nel significato a una domanda, confrontando i loro Embedding, invece di cercare le stesse identiche parole scritte: permette ad esempio di trovare un documento utile anche se non contiene nessuna delle parole usate nella domanda. → approfondito nella sezione 15 (Intelligenza artificiale).

Rischio

Un evento incerto che, se si verifica, può avere un impatto positivo o (più spesso) negativo su un progetto; identificarlo e pianificarne la gestione è uno dei compiti principali del project manager. → approfondito nella sezione 8 (Project Management).

Rollback

L'operazione di tornare indietro a una versione precedente e funzionante del software dopo che un rilascio ha causato problemi in produzione; il rollback del codice è di norma semplice, quello dei dati può non esserlo, se una modifica alla struttura del database è irreversibile. → approfondito nella sezione 10 (CI/CD) e nella sezione 14 (Ambienti di sviluppo). Si collega a: **Deploy/Rilascio** (l'operazione precedente, che a volte va corretta) e **Monitoring** (che ne segnala spesso la necessità).

SaaS

Sigla di Software as a Service: un modello cloud in cui usi direttamente un'applicazione già pronta tramite browser (come una webmail), senza doverti preoccupare di server, installazioni o aggiornamenti. → approfondito nella sezione 12 (Cloud).

Scalabilità verticale e orizzontale

La capacità di un sistema di gestire più carico di lavoro: la scalabilità verticale significa potenziare la singola macchina (più CPU, più RAM), quella orizzontale significa aggiungere più macchine che si dividono il lavoro. → approfondito nella sezione 12 (Cloud).

Scope (ambito)

L'insieme di tutto ciò che un progetto deve produrre (e, implicitamente, tutto ciò che invece resta fuori). Definire bene lo scope all'inizio, e proteggerlo con le Change Request, è uno dei compiti più importanti del project management tradizionale. → approfondito nella sezione 8 (Project Management). Si collega a: **WBS (Work Breakdown Structure)** (lo strumento con cui lo scope viene scomposto in attività) e **Scope Creep** (il rischio che corre se non viene controllato).

Scope Creep

L'espansione incontrollata dell'ambito (scope) di un progetto, quando si aggiungono via via piccole richieste "non concordate ufficialmente" senza passare da una Change Request, finché il progetto finisce per fare molto più di quanto pianificato, con tempi e costi che ne risentono. → approfondito nella sezione 8 (Project Management). Si collega a: **Change Request (richiesta di modifica)** (il meccanismo che serve a prevenirlo) e **Scope (ambito)** (ciò che si espande).

Scrum Master

La persona che facilita l'applicazione di Scrum nel team, rimuove gli ostacoli che intralciano il lavoro e protegge il team da interferenze esterne, senza dare ordini diretti su cosa fare. → approfondito nella sezione 6 (Scrum).

Scrumban

Un approccio ibrido che combina elementi di Scrum (come i ruoli e le cerimonie) con elementi di Kanban (come il flusso continuo e i limiti di WIP), utile per team che vogliono più struttura di Kanban ma più flessibilità di Scrum. → approfondito nella sezione 7 (Kanban).

Self-service

La possibilità di ottenere un ambiente, un database o una pipeline senza aprire un ticket e senza aspettare una persona: un modulo, un comando, un catalogo di opzioni pre-approvate, non un'email a chi "se ne occupa". → approfondito nella sezione 12 (Cloud). Si collega a: **Golden path** (il percorso che il self-service rende disponibile senza attriti) e **IDP (Internal Developer Platform)** (lo strumento che lo implementa).

Semantic Versioning

Una convenzione per numerare le versioni del software nel formato MAJOR.MINOR.PATCH (es. 2.4.1), dove ogni numero comunica il tipo di cambiamento avvenuto rispetto alla versione precedente. → approfondito nella sezione 3 (Come nasce un software).

Serverless

Un modello cloud in cui scrivi solo il codice della funzione che ti serve e il fornitore si occupa di tutto il resto (server, scalabilità, manutenzione), facendoti pagare solo per l'effettivo utilizzo. → approfondito nella sezione 11 (Architetture software).

Sharding

La strategia di scaling che divide i dati in porzioni indipendenti (shard) distribuite su macchine diverse, in base a una chiave di sharding (es. user_id o region): a differenza della Replication, ogni shard contiene solo una parte dei dati, non l'intero database, ed è la tecnica usata quando i dati o le scritture sono troppo grandi per una singola macchina. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Replication** (la strategia complementare, per scalare le sole letture) e **CAP Theorem** (il compromesso teorico che governa questi sistemi distribuiti).

Single Sign-On (SSO)

Un'autenticazione unica valida per molte applicazioni: chi si autentica lo fa una sola volta e da lì accede a email, repository, gestionale e strumenti cloud senza rifare il login su ciascuno. Concentra però anche il rischio: se il sistema di SSO cade, cade l'accesso a tutto contemporaneamente. → approfondito nella sezione 13 (Sicurezza). Si collega a: **Active Directory (AD)** (la directory su cui tipicamente si costruisce) e **Token** (lo strumento con cui l'identità viene poi dimostrata a ogni applicazione).

Sistema Operativo

Il software di base che gestisce le risorse del computer (CPU, RAM, Disco) e permette a te e alle altre applicazioni di usarle, facendo da intermediario tra hardware e programmi. Esempi: Windows, macOS, Linux. → approfondito nella sezione 2 (Fondamenti di informatica).

SLA

Sigla di Service Level Agreement: un accordo, spesso contrattuale, che definisce il livello di servizio garantito da un fornitore (ad esempio il tempo massimo di inattività accettabile o il tempo massimo di risposta a una segnalazione), con conseguenze concrete (spesso economiche) se non viene rispettato. → approfondito nella sezione 13 (Sicurezza).

Solution Architect

Il ruolo che tiene insieme una soluzione tra più sistemi e più team, traducendo requisiti di business in scelte tecniche e viceversa, e presidiando i requisiti non funzionali del sistema nel suo complesso — a differenza del Tech Lead, che si occupa di un singolo team, o dell'Enterprise Architect, che si occupa di tutta l'organizzazione. → approfondito nella sezione 8 (Project Management). Si collega a: **ADR (Architecture Decision Record)** (lo strumento con cui documenta le decisioni tra sistemi) e **Requisiti (funzionali e non funzionali)** (quelli non funzionali, che più spesso presidia).

SPI (Schedule Performance Index)

Un indice dell'Earned Value Management che misura quanto il progetto è in linea con i tempi pianificati: si calcola dividendo il valore del lavoro effettivamente completato (EV) per il valore del lavoro che era pianificato a questo punto (PV). Un SPI sopra 1 significa che si è avanti rispetto al piano, sotto 1 significa che si è in ritardo. → approfondito nella sezione 8 (Project Management). Si collega a: **EVM (Earned Value Management)** (il sistema di cui fa parte) e **CPI (Cost Performance Index)** (l'indice analogo sui costi).

Sponsor

La persona (o il ruolo) che finanzia il progetto e ne autorizza formalmente l'avvio, tipicamente firmando il Project Charter; è il principale punto di riferimento a cui il project manager rende conto sull'andamento generale. → approfondito nella sezione 8 (Project Management). Si collega a: **Project Charter** (il documento che firma) e **Stakeholder** (di cui è una figura specifica, con potere e interesse tipicamente molto alti).

Sprint

Un periodo di tempo fisso e breve (di solito 1-4 settimane) durante il quale il team Scrum lavora per completare un insieme di attività scelte dal Product Backlog, prodotto poi come Increment. → approfondito nella sezione 6 (Scrum). Si collega a: **Product Backlog** (da cui trae le attività) e **Increment** (il risultato concreto che produce).

Sprint Backlog

L'elenco delle attività selezionate dal Product Backlog che il team si impegna a completare durante lo Sprint corrente. → approfondito nella sezione 6 (Scrum).

Sprint Planning

L'incontro all'inizio di ogni Sprint in cui il team decide cosa realizzare, selezionando le voci dal Product Backlog e definendo come portarle a termine. → approfondito nella sezione 6 (Scrum).

Sprint Retrospective

L'incontro alla fine di ogni Sprint in cui il team riflette su come ha lavorato (non su cosa ha costruito) per individuare cosa migliorare nel prossimo Sprint. → approfondito nella sezione 6 (Scrum).

Sprint Review

L'incontro alla fine di ogni Sprint in cui il team mostra agli stakeholder ciò che ha completato, raccogliendo feedback utile per i prossimi passi. → approfondito nella sezione 6 (Scrum).

SQL

Sigla di Structured Query Language: il linguaggio usato per "interrogare" un Database relazionale, ad esempio per chiedere, aggiungere o modificare dati nelle tabelle. → approfondito nella sezione 2 (Fondamenti di informatica).

SRE (Site Reliability Engineering)

Un approccio, nato in Google, che applica pratiche e mentalità da ingegneria del software alla gestione dei sistemi in produzione, con l'obiettivo di renderli affidabili in modo misurabile (ad esempio tramite obiettivi di affidabilità concordati) invece che "a sensazione". → approfondito nella sezione 9 (DevOps).

Staging

Un ambiente che replica il più possibile le condizioni reali della Produzione, usato come ultima verifica prima del rilascio definitivo agli utenti. → approfondito nella sezione 14 (Ambienti di sviluppo).

Stakeholder

Qualsiasi persona o gruppo che ha un interesse nel progetto o ne è influenzato: può essere un cliente, un utente finale, un manager o un membro del team. → approfondito nella sezione 8 (Project Management).

Story Point

Un'unità di misura relativa (non temporale) usata per stimare quanto sia complessa una User Story rispetto alle altre, tenendo conto di difficoltà, incertezza e quantità di lavoro. → approfondito nella sezione 6 (Scrum).

Sub-task

In Jira, un pezzo di lavoro dentro una Story o un Task più grande, non stimabile da solo come issue a sé stante: il livello più fine della gerarchia Epic → Story/Task → Sub-task. → approfondito nella sezione 8 (Project Management). Si collega a: **User Story** (il livello sopra, di cui il sub-task è parte) e **WBS (Work Breakdown Structure)** (l'equivalente concettuale nel vocabolario PMP).

Tag

Un'etichetta che Git può assegnare a un Commit specifico, tipicamente per marcare un punto importante come una Release (es. "v2.4.1"). → approfondito nella sezione 4 (Git e GitHub).

TCP/IP

La famiglia di regole (protocolli) che permette ai computer di tutto il mondo di scambiarsi dati su internet in modo ordinato e affidabile, anche passando per reti diverse. → approfondito nella sezione 2 (Fondamenti di informatica).

Thread

Una "sotto-unità" di un Processo che può eseguire istruzioni in modo indipendente; più thread nello stesso processo permettono a un programma di fare più cose contemporaneamente in modo più efficiente. → approfondito nella sezione 2 (Fondamenti di informatica).

Throughput

La quantità di lavoro (attività completate, richieste gestite) che un team o un sistema riesce a portare a termine in un'unità di tempo: per un team è quante attività completa a settimana, per un sistema informatico è quante richieste gestisce al secondo. → approfondito nella sezione 7 (Kanban).

TLS/SSL (handshake)

Il protocollo che rende sicura una connessione HTTPS: prima di scambiare dati, client e server negoziano una chiave di cifratura condivisa (usando crittografia asimmetrica, più lenta ma solo per questa fase iniziale), verificano l'identità del server tramite un certificato firmato da una Autorità di Certificazione, e poi passano a cifrare tutta la comunicazione con crittografia simmetrica, molto più rapida. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **HTTPS** (il protocollo che questo handshake rende possibile) e **Token** (che viaggia in sicurezza proprio grazie a questa cifratura).

Token

Il termine ha due significati distinti nel corso. In ambito AI: la più piccola unità di testo (una parola, parte di una parola o un simbolo di punteggiatura) in cui un LLM scompone il testo per elaborarlo; i servizi di AI generativa spesso fatturano l'utilizzo proprio in base al numero di token letti e generati (→ sezione 15, Intelligenza artificiale). In ambito sicurezza: un "biglietto" temporaneo con scadenza breve che un sistema rilascia dopo l'autenticazione (tipicamente via SSO), da presentare a ogni richiesta successiva invece della password; va trattato come una password, e se finisce per errore in un repository va

considerato compromesso e sostituito (→ sezione 13, Sicurezza). Si collega a: **LLM (Large Language Model)** (che elabora il testo nel primo senso) e **SSO (Single Sign-On) / OAuth** (con cui si rilascia e usa nel secondo senso).

Training (addestramento)

Il processo con cui un modello di intelligenza artificiale “impara” analizzando grandi quantità di dati di esempio, aggiustando progressivamente i propri parametri interni finché le sue previsioni non diventano sufficientemente accurate. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Modello (AI)** (il risultato di questo processo) e **Overfitting** (un rischio tipico se il training non è condotto con attenzione).

Trigger

L'evento che fa scattare automaticamente l'avvio di una Pipeline, ad esempio un nuovo Commit su un branch o l'apertura di una Pull Request. → approfondito nella sezione 10 (CI/CD).

Triplo vincolo (triple constraint)

Il principio secondo cui un progetto è sempre limitato da tre fattori collegati tra loro — tempo, costo e ambito (scope), talvolta con la qualità come quarto vincolo aggiuntivo — e non si può cambiarne uno senza avere un effetto sugli altri: fare di più (scope) senza più tempo o più budget non è quasi mai possibile. → approfondito nella sezione 8 (Project Management). Si collega a: **Scope (ambito)** (uno dei tre vertici del triangolo) e **Change Request (richiesta di modifica)** (lo strumento con cui si negozia consapevolmente uno spostamento tra i tre vincoli).

Trunk Based Development

Una pratica in cui tutti gli sviluppatori integrano il proprio codice molto frequentemente su un unico ramo principale (il “trunk”), evitando Branch di lunga durata e favorendo una forte automazione dei test. → approfondito nella sezione 4 (Git e GitHub).

TTL (Time To Live)

Il tempo dopo il quale un dato salvato in cache scade automaticamente ed è considerato non più valido: semplice da implementare, ma per tutta quella finestra di tempo il dato in cache può non essere più aggiornato rispetto al database. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Cache-Aside (lazy loading)** (la strategia di caching con cui questo meccanismo di scadenza viene usato più spesso).

User Story

Una breve descrizione di una funzionalità scritta dal punto di vista dell'utente, spesso nel formato “Come [ruolo], voglio [obiettivo], per [beneficio]”, usata per catturare i requisiti in modo semplice e centrato sulle persone. → approfondito nella sezione 6 (Scrum). Si collega a: **Requisiti (funzionali e non funzionali)** (da cui spesso nasce) e **Product Backlog** (dove viene inserita in attesa di essere pianificata).

Vault (dei segreti)

Uno strumento dedicato a custodire in modo cifrato password, chiavi di accesso e altre credenziali, concedendone l'uso solo a chi (una persona o un sistema, come una pipeline) ne ha davvero bisogno, invece di scriverle nel codice o passarle a mano tra le persone del team. → approfondito nella sezione 14 (Ambienti di sviluppo).

Velocity

La quantità media di Story Point che un team Scrum riesce a completare in uno Sprint, misurata negli Sprint passati e usata per stimare quanto lavoro pianificare in futuro. → approfondito nella sezione 6 (Scrum).

Versioning

La pratica di assegnare un numero o un'etichetta univoca a ogni versione del software, per poter distinguere, confrontare e tornare a versioni precedenti quando necessario. → approfondito nella sezione 3 (Come nasce un software).

Virtual Machine (VM)

Un "computer dentro il computer": un ambiente simulato dal software che si comporta come una macchina indipendente, con proprio sistema operativo, pur condividendo l'hardware fisico con altre VM. → approfondito nella sezione 2 (Fondamenti di informatica).

Vista (VIEW)

Una query SQL salvata con un nome, che si comporta come se fosse essa stessa una tabella: permette a chi la usa di non dover riscrivere (né conoscere) la JOIN che c'è dietro. Non elimina il costo della query sottostante, che viene comunque eseguita a ogni chiamata. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **JOIN** (l'operazione che tipicamente racchiude) e **Database relazionale** (il contesto in cui vive).

Waterfall

Un approccio tradizionale alla gestione dei progetti in cui le fasi (analisi, progettazione, sviluppo, test, rilascio) si susseguono in modo lineare e sequenziale, una dopo l'altra, a differenza dell'iteratività di Agile. → approfondito nella sezione 5 (Agile).

WBS (Work Breakdown Structure)

La scomposizione gerarchica di tutto il lavoro necessario a completare lo scope di un progetto, dai macro-obiettivi fino a singole attività gestibili e stimabili, spesso rappresentata con codici gerarchici (1, 1.1, 1.1.1...) anche in un semplice foglio Excel. Il livello più basso della scomposizione si chiama **work package**: abbastanza piccolo da poter essere stimato con sicurezza e assegnato a una singola persona. È uno degli strumenti cardine della pianificazione tradizionale, e serve da base per stimare

tempi e costi. → approfondito nella sezione 8 (Project Management). Si collega a: **Scope (ambito)** (ciò che la WBS scompone) e **Product Backlog** (l'equivalente Agile, più flessibile e continuamente ri-prioritizzato invece che fissato all'inizio).

WIP (Work In Progress)

Il numero di attività che sono attualmente “in corso” in una Board Kanban; limitare il WIP (fissando un tetto massimo per colonna) aiuta il team a concentrarsi e a completare il lavoro più rapidamente invece di iniziarne troppo in parallelo. → approfondito nella sezione 7 (Kanban).

XML

Sigla di eXtensible Markup Language: un formato di testo, più “verboso” di JSON, usato per organizzare e scambiare dati tramite tag simili a quelli dell'HTML. → approfondito nella sezione 2 (Fondamenti di informatica).

Collegamenti

- [17. Piano di studio](#) — un percorso suggerito per ripassare tutti questi concetti in modo organico

Risorse

- [Glossario Agile Alliance](#) — glossario dei termini Agile e Scrum (in inglese)
- [GitHub Docs](#) — documentazione ufficiale, utile per verificare la terminologia esatta usata dalla piattaforma (Pull Request, Issue, Actions...)
- [Wikipedia — Glossario di informatica](#) — per approfondire i termini di base